ChatGPT

Jira as a Graph Database – Proposal for Atlassian Executives

Executive Summary

Dinis Cruz is a seasoned technology leader and Atlassian expert with over a decade of experience pushing Jira beyond traditional use cases. He has pioneered research and practical implementations of **using Jira as a Graph Database**, where each Jira issue serves as a "node" and each issue link represents a relationship "edge." Through years of experimentation and consulting, Dinis has developed a clear vision: Jira's built-in issue linking and customization can model complex networks of information – effectively turning Jira into a powerful knowledge graph. In recent months, he has **validated this vision with in-depth research** (aided by AI) and open-source tooling: - **Long-Standing Jira Expertise:** Former CISO of Photobox Group, Dinis refactored security and risk workflows to treat Jira as a graph-backed system, enabling data-driven decisions [1]. He has consistently leveraged Jira in innovative ways since 2016, sharing insights at OWASP and DevSecCon conferences. - **Pioneering Graph Research:** Dinis's original research (summarized in his LinkedIn post "Using Jira as a Graph DB") articulates Jira's unique value as a graph database. He demonstrates how Jira issues (nodes) and link types (edges) can form rich entity-relationship models, and compares Jira's capabilities to dedicated graph databases like Neo4j and Amazon Neptune [2]. Notably, many limitations identified in using Jira as a graph have known solutions in Dinis's work. - **Innovative Open-Source Tools:** Dinis has developed tooling to augment Jira's graph potential – for example, a Jira synchronization system ("JSync") that exports Jira data to JSON, S3 and Git for offline graph queries [3], an experimental **Project Lumos** connector that integrates Jira with external graph databases [4], and **MGraph**, a memory-first graph database for AI and serverless apps [5]. These projects add graph querying and visualization capabilities on top of Jira. - **Strategic Collaboration Opportunity:** Dinis's work represents a timely opportunity for Atlassian. By partnering with him, Atlassian can **unlock new Jira capabilities** (advanced analytics, knowledge graphs, AI integrations) and drive market differentiation. This proposal outlines a collaboration where Atlassian supports and invests in Dinis's vision – potentially incorporating his graph techniques and tools into the Jira platform. The result would be mutual benefit: Atlassian gains cutting-edge features and thought leadership, while Dinis's innovations reach a broader audience with Atlassian's backing.

In summary, Dinis Cruz offers Atlassian a unique combination of deep Jira expertise and a forward-looking vision to treat Jira as a graph database. His proven research and tools can accelerate Atlassian's own roadmap, especially in areas of AI and knowledge management. A strategic collaboration or investment would position Jira at the forefront of modern, graph-powered work management, strengthening Atlassian's competitive edge.

Dinis Cruz's Background and Vision

Background: Dinis Cruz is a well-respected figure in the software and security community, known for his leadership in organizations and open-source projects. He served as CISO of Photobox Group, where he championed innovative uses of Atlassian Jira and Confluence to improve security operations. At Photobox (2017–2019), Dinis's team fundamentally rethought how Jira was used: they transformed the company's risk tracking and project workflows into a **graph model**. In a 2019 OWASP London presentation, he explained how his organization "refactored their risk workflow and Jira implementation

to represent data as a graph stored in a queryable database," using custom tools and visualizations [1]. This early success demonstrated his core belief – that **the only effective way to manage and understand complex, relational data is as a graph** [1].

Throughout the late 2010s, Dinis evangelized the graph approach in various forums. In a 2018 "Thinking in Graphs" talk, he showed how many domains (security incidents, software projects, organizational structures) can be modeled as graphs, highlighting that his company was using **Neo4j and Jira as graph databases to model security projects and incidents** [6]. He even reached out to Atlassian on social media around that time, noting that at Photobox they were "using Jira as a Graph DB to manage risk data and workflows," and looking to exchange ideas – underscoring his role as a pioneer in this space.

Vision: Dinis's vision is to unlock Jira's latent power as a graph-based knowledge hub for organizations. He views Jira not just as an issue tracker, but as a **rich graph of the organization's knowledge, work, and people**. Every issue can be a node representing an entity (e.g. a task, a requirement, a person, a risk), and every link or relationship captures valuable context (dependencies, ownership, impacts). Over years of research and practice, Dinis has developed a structured approach to realize this vision: - **Graph Modeling in Jira:** Rather than using Jira's default project/issue setup in a purely task-centric way, Dinis restructures Jira to behave like a graph database. In his methodology, **each Jira project corresponds to a node type**, and each issue within is an instance of that entity type [7]. For example, one project might hold "Asset" nodes, another "Risk" nodes, another "Control" nodes, etc. Jira's native issue linking is then used to define relations between these entities (edges), effectively creating a living ontology of the business. He emphasizes a one-issue-type-per-project strategy – "each project represent[s] a distinct node type in a graph" [8] – which brings clarity and consistency to the data model. - **Semantic Links (Edges):** A key part of Dinis's vision is leveraging Jira's **issue link types** to encode semantics for relationships. Jira allows custom link definitions with an outgoing and incoming description (for example, a link type could be defined as "*mitigates*" in one direction and "*mitigated by*" in the reverse). Dinis notes that Atlassian's design of requiring both an outward and inward description for each link is brilliant – **"when used as described, this structure enables the creation of powerful ontologies and taxonomies"** [9]. In other words, Jira issues can be linked in meaningful ways (Project A *"depends on"* Project B, Task X *"blocks"* Task Y, Risk R *"is mitigated by"* Control C, etc.), and those links are first-class data. This turns Jira's web of issues into a true graph network of knowledge. - **Visualization and Querying:** Dinis's vision includes making these implicit graphs visible and queryable. At Photobox his team dumped Jira data into Elastic and used Slack-bot commands and PlantUML scripts to query relationships and generate diagrams [10] [11]. More recently, he has used tools like ChatGPT to dynamically map Jira data. The goal is to let users traverse the graph: find all paths of influence from a CEO's strategic goal down to individual tasks, or aggregate all risks impacting a particular product, and so on – tasks that a graph model handles naturally but are hard with flat issue lists.

Crucially, Dinis's vision aligns with trends in enterprise knowledge management. He believes that **Jira can serve as a central knowledge graph for an organization's projects and processes**, especially when combined with Confluence and other Atlassian data. In his own words, Jira is an "amazing graph database," and he has lamented that "there hasn't been much evolution in the use of Jira as a Graph Database," calling it a *missed opportunity* [12]. He advocates for Atlassian to invest more in this area to unlock "incredible value from Jira" [12]. The rest of this proposal will detail Dinis's work to date and how Atlassian can strategically capitalize on it.

Timeline of Key Jira–Graph Initiatives

- **2016 – Early Experiments:** Dinis begins leveraging Jira in unconventional ways while working in security roles. At an OWASP AppSec EU 2016 talk, he introduces the idea of using Jira to manage security risks as interconnected data points rather than just tickets (foreseeing Jira's graph potential). This set the stage for deeper graph modeling in the years ahead.

- **2017 – "Graph-Based Organization" Advocacy:** As CISO at Photobox, Dinis leads the concept of a "graph-based security organization." He presents a keynote at DevSecCon London 2017 describing how security data and workflows can be mapped as graphs with Jira and other tools, to better visualize how security initiatives link to business objectives. This helped socialize the value of graphs to both technical and executive stakeholders.

- **2018 – Thinking in Graphs (Photobox):** By 2018, Dinis's team formally implements graph principles in Photobox's Jira. In June 2018 he delivers *Thinking in Graphs v1.0*, showcasing how **Neo4j and Jira were used in tandem as graph databases to model security projects and incidents** [6]. Around this time, he also collaborates at the Open Security Summit on using Jira data with graph visualization (Neo4j/Neovis) tools. These efforts prove that even complex security and DevOps processes (incidents, changes, risk assessments) can be represented and managed as nodes and links in Jira.

- **2019 – Photobox Risk Graph Project:** Dinis and co-presenter Maeve Scarry share the *Creating a Graph-Based Security Organisation* case study at the OWASP London chapter (April 2019). This highlighted a major milestone: **Photobox's risk management workflow was refactored so that Jira captured a graph of risks, controls, assets, and people**. All relationships (like which controls mitigate which risks, which team owns which control, etc.) were tracked via Jira issues and links. The team even built a serverless pipeline exporting Jira data to a queryable datastore (ElasticSearch) and visualized the graph with PlantUML and custom dashboards [10] [11]. The result was a living risk knowledge graph that empowered data-driven decisions [10] – a pioneering achievement for Jira usage. (Notably, Slide 12 of that presentation plainly states: **"We use JIRA as a Graph Database."**)

- **2020 – 2021 – Continued Refinement:** In subsequent years, Dinis continued to refine his approach through consulting projects. He began developing automation bots (under the OWASP "OSBot" initiative) to interface with Jira and other systems. During this period, he laid conceptual groundwork for tools to sync Jira data out to graph databases or vice versa, anticipating the need to overcome Jira's reporting/query limitations.

- **2022 – OSBot-Jira and Prototypes:** Dinis open-sourced **OSBot-Jira**, a toolkit for automating Jira operations and extracting data (as part of the OWASP Security Bot project). This included prototypes of **Jira sync scripts** that could pull all issue data (including links, history, etc.) and push into external storage for graph analysis. The ideas behind "JSync" were born – treating Jira as the authoritative source, but replicating its data into formats better suited for complex queries or visualization.

- **Late 2024 – "Jira as Graph DB" Research:** With the emergence of advanced AI (OpenAI's ChatGPT), Dinis undertook a comprehensive research project to document and validate his Jira graph approach. In a LinkedIn post titled *"Using Jira as a Graph DB"* (posted around early 2025), he shared a **detailed report (co-written with ChatGPT) that compiled knowledge on Jira as a graph database**. This report not only cited prior examples (including his own Photobox case) but

also analyzed Jira's capabilities against purpose-built graph databases and discussed best practices. One highlight is that **Jira's core strengths (flexible issue types, customizable link semantics, built-in UI for workflows) make it a surprisingly effective graph data store, within certain scale limits** [2] .

- **Early 2025 – Publication of Key Documents:** Building on the initial report, Dinis published a series of technical documents (as PDFs on LinkedIn) delving into specific aspects of Jira as a graph. These include:

- *"Jira's Unique Value as a Graph Database"* – a thorough analysis of Jira's graph strengths, including a **business case and technical comparison** of Jira vs. Neo4j, AWS Neptune, ArangoDB, etc., and identification of Jira's limitations (most of which Dinis has workarounds for) [2] .
- *"Graph Database Techniques in Jira – Issue Type and Project Design"* – a technical guide to Dinis's node modeling approach (one issue type per project, each project as a node category) [7] .
- *"... – Workflow Design and Customization"* – guidance on designing Jira workflows to support graph use-cases, emphasizing simplicity and the *"principle of least surprise"* (workflows should make it easy to do the right thing) for users [13] .
- *"... – Issue Link Types"* – documentation of how to define and use custom link types to represent domain-specific relationships (for example, in a risk management graph: Issue A **"mitigates"** Issue B) and how this built-in linking mechanism in Jira underpins powerful ontologies [9] .
- *"Storing Timestamps in Graph Databases: A Scalable and Semantic Approach"* – a research piece on handling temporal data in graph structures [14] . This indicates Dinis's attention to advanced topics (like capturing time-based events or changes in Jira, and representing those in a graph).

Each of these documents was curated as part of his "Using Jira as a Graph DB" collection and demonstrated not just theoretical knowledge but practical solutions and examples. They serve as a knowledge base that Atlassian could draw upon for product improvement.

- **Q1 2025 – First Visualizations & Project Lumos:** Dinis also shared live examples of the concept in action. He posted **visualizations of Jira data as graphs** (using PlantUML and other tools) – for instance, the "paths from CEO" graph that traced how work items flow downward from a CEO through various roles to individual tasks. Such posts showed the tangible value of graph views (revealing chains of command and dependencies). Furthermore, Dinis drafted **Project Lumos**, a working-backwards style proposal for a serverless Jira-to-GraphDB connector [4] . Project Lumos envisioned a small investment (e.g. an example £8k) to build an open-source bridge between Jira and an enterprise graph database, complete with a mock press release announcing how it "unlocks hidden insights from Jira" by syncing data into a graph backend [15] [16] . The timing and content of Project Lumos suggest Dinis is actively seeking to **collaborate with industry (possibly Atlassian or graph DB vendors) to operationalize his ideas**.

- **Today:** Dinis is continuing this work as an independent innovator. He is actively engaging the community (and tagging Atlassian staff on social media) to spark interest. His **vision for Jira as a Graph DB is fully formed and supported by prototypes**. He is now looking to partner with Atlassian to bring this to fruition at a larger scale. The next sections will highlight the core findings of his research and the strategic benefits for Atlassian.

Highlights of Dinis Cruz's Graph Research in Jira

Dinis's recent research provides a comprehensive look at why and how Jira can function as a graph database. Key highlights from his findings include:

1. Jira's Unique Advantages as a Graph Store: Despite not being labeled a graph database, Jira has intrinsic features that make it surprisingly powerful for graph-structured data: - **Flexible Issue Types & Projects:** Jira lets users define custom issue types and fields, and organize issues into projects. Dinis capitalizes on this by assigning **each project a specific entity type**, thereby achieving an *implicit schema* for the graph [7]. For example, one can have a "Roles" project for people/roles, a "Systems" project for applications, a "Risks" project, etc. Each issue in those projects is a node instance, and its fields describe node attributes. This approach is detailed in his document *Graph Database Techniques in Jira – Issue Type and Project Design*, which describes structuring Jira in a way that "issues act as nodes and links between them form relationships – much like an entity-relationship model or a knowledge graph" [7]. By using a one-node-type-per-project convention, the data becomes **self-descriptive and easier to navigate**, avoiding the clutter of multiple issue types mixed together [8]. - **Rich, Bidirectional Relationships:** Perhaps Jira's most graph-centric feature is its issue linking system. Jira allows definition of link types with both an outward and inward description (for instance, *"blocks"* vs *"is blocked by"*, *"relates to"* vs *"relates to"* (reciprocal), or custom ones like *"parent of"* / *"child of"*). Dinis identifies this as a **unique strength**: *"One of the design patterns Atlassian implemented… was requiring every edge (i.e., Jira link) to have both an outgoing and an incoming description. When used … this enables the creation of powerful ontologies and taxonomies."* [9] In essence, each link in Jira carries semantic meaning in both directions. This is a feature many graph databases themselves implement for data modeling, and Jira provides it out-of-the-box for work items. Dinis's research shows that by thoughtfully defining custom link types (aligned to business relationships), one can represent complex knowledge graphs in Jira. For example, in a project portfolio graph, a "implements -> is implemented by" link might connect a high-level initiative issue to specific feature issues. In an org chart graph, a "manages -> is managed by" link can connect a manager (a Jira issue in a "Role" project) to their direct report (another "Role" issue). **These semantics turn Jira from a simple tracker into a true graph of nodes and edges with meaning**. - **User-Friendly Interface and Workflow:** Unlike typical graph databases, Jira comes with an entire user interface, workflow engine, and permissions model. Dinis emphasizes that Jira's UI and workflow capabilities are actually a differentiator when using it as a graph DB. Non-technical users can create and edit "nodes" (issues) via forms, use Jira's built-in screens to populate attributes, and transition items through workflows. In his *Workflow Design and Customization* notes, he highlights that Jira workflows can enforce data integrity and guide user behavior (using a "principle of least surprise" – making it easy to do the right thing) [13]. This means the graph data can be kept consistent and clean as people interact with it. For example, a workflow can ensure that if a Risk node is marked "Mitigated", a link to at least one Control node is required (enforcing a relationship before closure). Traditional graph databases lack this kind of user-facing functionality – a major advantage for Jira in practical enterprise use. - **Integration with Confluence & Ecosystem:** While not explicitly detailed in the LinkedIn posts, it's implied in Dinis's vision that Atlassian's ecosystem (Confluence pages, Bitbucket, etc.) could augment the Jira graph. For instance, Confluence pages could be linked to Jira issues to attach documentation nodes to the graph, or Bitbucket commits linked to story issues to connect code to requirements. Atlassian has an entire suite that, if treated as a graph, would yield a **unified knowledge graph of development and project data**. Dinis's work primarily showcases Jira, but the principles extend to a broader Atlassian graph of work.

2. Research Backed by AI and Market Analysis: Dinis did not rely solely on personal anecdote – he systematically researched the topic. After documenting his own experiences (including a voice-recorded brainstorm of a decade of using Jira as a graph), he used ChatGPT's "Deep Research" feature to gather external references and articulate a formal report [17]. The resulting **"Jira's Unique Value as a Graph Database" report** is thorough: it even includes comparisons between Jira and dedicated graph databases like Neo4j, AWS Neptune, and ArangoDB [18]. According to Dinis, *"ChatGPT was able to articulate the business and technical case for using Jira as a Graph DB… It also includes a market comparison with other Graph DBs… Most of the limitations identified… are ones for which I have found effective solutions."* [2]. This is a crucial point – the research acknowledges, for example, that Jira is not optimized

for large-scale graph queries or certain graph algorithms. However, Dinis already has approaches to mitigate these limitations (as we will see in his tooling). By citing industry knowledge, he strengthened the credibility of using Jira in this non-traditional way, which would be important to Atlassian executives considering such an angle. The fact that one of the key examples the AI found was Dinis's own 2018 presentation [19] speaks to how early he has been in this space, and how little others have published – reinforcing that this is *pioneering work*.

3. Practical Examples and Visuals: To make the research tangible, Dinis's posts include examples of Jira-as-graph in action: - **PlantUML Graphs from Jira Data:** One LinkedIn update shows a snippet of a graph diagram generated via PlantUML from Jira data, accompanied by Dinis's remark: *"Creating graphs like this (from Jira data using PlantUML) feels like reconnecting with an old friend... we've been through a lot"* [20]. The image he shared (see Figure below) was generated by exporting a set of Jira issues and their links and feeding them to a PlantUML script. This produced a **graph visualization of the Jira dataset**, demonstrating that with a bit of scripting, one can get an actual graph diagram out of Jira information. Such visualization could show, for instance, how a higher-level Epic breaks down into Stories and Tasks and how those tasks relate to components or teams. Dinis noted that ChatGPT-4 assisted in creating some of these models, and even complimented the model as one of the best seen [21] – indicating that AI tools found the structure logical and well-formed.

【54†embed_image】 *Figure: An example schema from Dinis Cruz's Jira-as-Graph approach. Each node type (Role, Product, Feature, Story, Task, etc.) is represented as a dedicated Jira project (one issue type per project), and custom issue link types define the relationships (edges) between nodes (e.g. "manages", "delivers", "blocks"). This diagram illustrates how Jira issues and links form a connected knowledge graph of an organization's hierarchy and work.* In Dinis's implementation, such a schema was not just theoretical – it was implemented in Jira to track real entities like team roles, software features, user stories, tasks, and even dependencies like blocked bugs. The structured approach ensures that queries like "what features does Product X have, and who is working on them?" become trivial graph traversals through the linked issues.

- **"Paths from CEO" Visualization:** Another striking example Dinis shared was a graph query for "paths from the CEO". Using a set of Jira issues representing company roles and linking them (CEO -> direct reports -> their reports, and so on, as well as linking those roles to the projects or epics they sponsor), he generated a visualization of the organizational flow from the CEO down to individual contributors. This first-of-its-kind Jira visualization shows how information or work streams propagate from the top of an organization to the execution levels【30†L29-L37】. While the LinkedIn post only briefly captions it, the implication is powerful: with Jira as a graph, one can ask questions like "show me all the work that originates from this executive" or "how are teams connected through shared goals?" – questions that typical project reports cannot easily answer.

- **Decision Graphs and Auto-Documentation:** In a discussion on capturing decisions, Dinis provided a real use-case: He described a method where architecture decisions were stored as a set of linked Jira issues (for Context, Decision, Status, Consequences), and then an **ADR (Architecture Decision Record) document was autogenerated as a PDF from those Jira issues**

【44†L31-L39】. *“In the past, I did exactly this using Atlassian Jira, where we used Jira as a graph database and autogenerated PDFs like this from the data in the respective Jira issues.”* 【44†L33-L39】. This example underscores how treating Jira as a graph can improve knowledge management – decisions were not just written in a static document; they were broken into structured nodes (facts, choices, outcomes) in Jira and linked. The master source of truth remained dynamic in Jira, and documents could be generated on the fly. This is a concrete illustration of moving beyond static record-keeping to living data. It also shows integration of Jira with content generation (which Atlassian could extend via Confluence or native PDF export capabilities). Executives reading this can appreciate that such an approach ensures institutional knowledge (like why a big decision was made) is captured in a queryable form, not lost in email or slide decks【44†L51-L59】.

In sum, Dinis’s research distills into a few core insights: **Jira already has the fundamental building blocks of a graph database** (nodes, edges, labels, basic queries via JQL, etc.), and with the right design and extensions, it can be used to model and query complex relationships in ways that benefit organizations. His documentation and examples provide a playbook on how to do this. It’s a novel idea that he has backed with evidence and working prototypes, making it a credible innovation for Atlassian to consider.

Extending Jira’s Graph Capabilities: Dinis’s Tools (JSync, Lumos, MGraph)

While Jira provides the foundation, Dinis recognized that unlocking its full graph potential requires some extensions and integrations. To that end, he has created or proposed several tools/projects that enhance Jira’s graph capabilities:

- **JSync – Jira Data Synchronization System:** One challenge with using Jira as a graph is performing advanced queries or analytics, since Jira’s built-in search (JQL) is not designed for multi-hop graph traversals. Dinis’s solution is to **export and synchronize Jira data to a graph-friendly datastore**. In a project plan entitled *“JIRA Exporter and Synchronization System”*, he outlines a method to pull all Jira issue data and keep it in sync with external storage like JSON files in S3 or a Git repository [23]. The exported data (in JSON) can then be loaded into graph databases or queried with graph query languages without impacting the live Jira system. The plan covers getting Jira data into a format where “we can query the data without hitting Jira servers” by using a combination of cloud storage and version control [24]. This essentially decouples heavy query workloads from Jira. Dinis notes this approach is something he has implemented “a couple of times before (and am about to do again with a current client)” [3], highlighting its practicality. In essence, **JSync** (as we’ll call it) acts as a bridge: Jira remains the source of truth for data entry and updates, but a mirror of the data lives in a graph-optimized environment for analysis. Atlassian could leverage this concept by offering an official data export or sync feature (perhaps streaming changes to an Atlassian-owned graph service or data lake). Doing so would enable powerful reporting and AI use-cases on Jira data without compromising Jira performance.

- **Project Lumos – Jira-to-GraphDB Connector:** Project Lumos is Dinis’s proposal to formalize the above syncing concept specifically for a third-party graph database platform. In his Working Backwards document for Lumos, he envisions an **open-source, serverless connector that**

automatically feeds Jira issue data into a chosen graph database (referred to generically as "GraphDB XYZ") [25]. The connector would run continuously (e.g., as AWS Lambda functions or similar), listen for updates in Jira (via webhooks or API polling), and update the graph database accordingly. The hypothetical press release for April 2025 that Dinis wrote paints the picture: *"PROJECT LUMOS: Serverless Jira-to-GraphDB XYZ Connector – Unlocking Hidden Insights from Jira Using GraphDB XYZ"*. It describes a small investment that yields a **reliable, automated bridge syncing Jira data into GraphDB deployments**, enabling teams to leverage their graph analytics on Jira data [15] [16]. Dinis emphasizes he has done this "many times before" in custom forms [26], giving confidence in feasibility. For Atlassian, this kind of connector could be transformational – it suggests a path for Jira to integrate with enterprise graph analytics tools. Rather than fearing that approach (it could be seen as offloading data to another platform), Atlassian could **embrace it as an integration strategy**. Supporting connectors to Neo4j, TigerGraph, AWS Neptune, or other graph databases would position Jira as *graph-friendly* and capable of playing in the growing world of enterprise knowledge graphs. It would effectively let Jira data participate in graph queries and AI reasoning in external systems. Atlassian might even develop its own managed graph service to pair with Jira (so that customers don't have to bring a separate graph DB). Project Lumos, as Dinis structured it, is lightweight (serverless, cloud-driven) and could be a starting blueprint for Atlassian to develop official offerings.

- **MGraph – Memory-First Graph Database for AI/Serverless Apps:** During his research, Dinis realized that existing graph databases also have complexities or may not be tailored for certain emerging needs (like on-demand serverless execution or in-memory processing for AI tasks). In response, he created **MGraph-DB (also referred to as MGraph-AI)** – a new open-source graph database optimized for ephemeral, in-memory operations and integration with AI workflows. He mentions in a LinkedIn article that *"MGraph-DB is the serverless graph database that I recently published which provides … the ability to easily manipulate and merge JSON objects as nodes and edges"* [5], calling it a "Memory-First Graph Database for GenAI and Serverless Apps" [27]. This tool is designed to be lightweight and to allow programmatically building graphs on the fly (for instance, to support a chatbot or an agent reasoning over a private knowledge graph). In practice, Dinis has used MGraph in a project (MyFeeds.ai) to build semantic knowledge graphs out of text and feed them to LLMs [28] [29]. The relevance to Jira is that MGraph could be used in tandem with Jira to do fast, on-demand graph analytics. For example, one could pull a subset of Jira issues (say all items related to a specific project or team), load them into MGraph in-memory, and run complex graph algorithms or AI queries (like "find expertise overlaps between these two teams based on issue assignments"). MGraph essentially fills a gap where neither Jira nor big graph databases might be ideal – it's a nimble tool for real-time graph computations. Atlassian might see strategic value in this if they are considering bringing more AI into Jira: a memory-first graph engine could be embedded in an Atlassian AI assistant to help answer questions by traversing relationships in Jira data on the fly. Given that Dinis has already built this with Atlassian use-cases in mind, Atlassian could consider supporting the MGraph project or adopting some of its approaches.

- **OSBot and Automation Tools:** In addition to the above, it's worth noting Dinis's **OSBot-Jira** project (from OWASP) which includes various automation scripts for Jira. These include using Jira's API to query issues and even update them from scripts or Jupyter notebooks. For instance, Dinis demonstrated updating Jira issue statuses directly from a Jupyter interface as part of a workflow, and generating graphs in notebooks. While not a single product, these automation capabilities show how Jira's graph can be interacted with programmatically. It aligns with Atlassian's recent emphasis on automation (e.g., Jira Automation rules) – Dinis's work could inspire more advanced automation that treats issue links as pathways to traverse for triggers

(like “if a task is blocked by an incident, notify the incident’s owner” – essentially graph traversal in automation rules).

Together, **JSync, Project Lumos, and MGraph provide a toolkit for maximizing Jira’s graph potential**. They address the main technical hurdles: extracting data for heavy queries, integrating with graph databases at scale, and enabling new graph-driven AI use cases. All these tools are open-source or proposed as such, which lowers the barrier for Atlassian to experiment with or support them. Dinis’s willingness to co-develop these with AI assistance (he notes using ChatGPT, Otter.ai, and Claude to refine his Project Lumos document [30]) also suggests a fast development cycle – small investments can yield functional prototypes quickly.

For Atlassian, adopting these ideas could mean: - Enhanced enterprise integrations (appealing to large customers who want to plug Jira data into broader analytics platforms). - New add-ons or features (e.g., an official Jira Data Graph Exporter, or a Graph View in Jira that uses an MGraph-like engine behind the scenes). - Strengthening the developer ecosystem (imagine encouraging Marketplace vendors to build graph analysis apps for Jira – Dinis’s work could be the blueprint).

In summary, Dinis’s tools fill in the gaps to make **Jira not just a static issue tracker, but a living graph data source**. They are enablers that Atlassian can collaborate on to bring this concept to production-grade reality.

Strategic Potential for Atlassian and Jira

Embracing Jira as a graph database is not just a technical curiosity – it has significant strategic implications for Atlassian. Here are the key opportunities and benefits:

1. A New Dimension of Capabilities in Jira: By leveraging graph concepts, Atlassian can add features to Jira that make use of relationships in smarter ways: - **Advanced Analytics & Insights:** Today, Jira can tell you the status of tasks and basic dependencies, but a graph-enabled Jira could answer far more complex questions. For example: *“What upstream work or components would be impacted if this epic is delayed?”* or *“Who in the organization has the most connections across projects (potentially a key person for knowledge transfer)?”* Graph queries can uncover hidden patterns (like bottlenecks or key influencers) that traditional filters cannot. Atlassian could build **visual dependency maps, impact analysis tools, and influence graphs** right into Jira, giving managers and executives a system-wide view of how work is connected. This is a strong differentiator, turning Jira into a decision support system, not just a task tracker. - **Knowledge Graph & Search:** With issues linked semantically, Jira becomes a **knowledge graph** of the organization’s work. This can improve search and discovery dramatically. Instead of keyword search, users could query the graph: *“Find all tasks related to Project X that involve both Backend and Security teams”* or *“Show the chain of requirements that led to feature Y being developed.”* Atlassian could integrate this into Jira’s UI as an intelligent search or query builder. It would set Jira apart as a platform that not only stores work items but *understands* their context. - **AI and Atlassian Intelligence:** Atlassian has been integrating AI (e.g., the announced Atlassian Intelligence features that summarize issues, answer questions, etc.). A graph-backed Jira would supercharge these AI capabilities. Large Language Models (LLMs) augmented with a Jira knowledge graph could answer questions like, *“What were the reasons behind last quarter’s downtime and which teams were involved in fixes?”* by traversing links between incident tickets, problem reports, and team owners. The structure provides **grounding for AI**, reducing hallucination because the AI can follow real relationships in data. Dinis’s approach with MGraph and AI demonstrates this potential – graphs make AI results more explainable and traceable [31] [32]. Atlassian could become a leader in “augmented project intelligence” by combining its data graph with AI assistants. - **Modeling Organizational Dynamics:** Companies often try to map how

information flows or how teams are structured (org charts, RACI matrices, etc.). Jira already contains a lot of this implicitly (who reports to whom on tasks, which teams collaborate on issues via links). Using Jira as a graph makes **organizational network analysis** possible. Atlassian could provide dashboards that reveal communication paths, silos, or highly interconnected projects. For leadership, this is invaluable: it turns Jira into a map of the organization's functional structure. For instance, Dinis's "paths from CEO" visualization showed an org chart of work at a glance – something executives greatly appreciate. By natively supporting such views, Jira could become not just a tool for project managers, but also for HR, strategy, and operations to understand their org.

Figure: A "Paths from CEO" graph generated from Jira data (as demonstrated by Dinis Cruz). This graph view traces how high-level goals and issues assigned to a CEO break down into sub-issues and tasks across the organization, ultimately reaching individual contributors. It highlights chains of accountability and influence that are usually buried in disparate tickets. By adopting graph visualizations like this, Atlassian could enable executives to instantly see how work flows through their company. This visualization underscores the rich relational data already present in Jira. Making such relationships visible helps in identifying key dependency points or overburdened roles (for example, if many critical tasks funnel through one manager, that risk can be spotted). It also celebrates the connectivity of teams – a selling point for Jira as a central collaboration hub.

2. Market Differentiation and Competitive Edge: Jira is already a market leader in project and issue tracking. However, competitors are emerging (e.g., Azure DevOps, Monday.com, Linear, etc.) that try to offer simpler or more specialized experiences. By infusing Jira with graph database capabilities, Atlassian can differentiate on a level others are not even approaching. No mainstream project management tool today markets itself as a **graph-based platform** that can manage complex, interrelated data natively: - Atlassian could position Jira as the tool not just for managing tasks, but for **managing knowledge and relationships** within projects. This appeals to sophisticated enterprise customers who are looking into building "digital twin" organizations or robust knowledge management systems. Instead of buying a separate knowledge graph solution, they might leverage what they already have in Jira. - Embracing this angle could also future-proof Jira. Work management is evolving – companies want interconnected views of their data (the rise of data lakes, big queryable datasets, etc.). If Jira can serve both as an operational system and a source of analytical insights (via graph queries), it remains central to both day-to-day users and data analysts. This beats tools that are siloed or require heavy export to analyze. - From a marketing standpoint, Atlassian could introduce new premium features or editions, for example **"Jira Enterprise Graph"** – essentially an offering that includes graph database integration, advanced relationship mapping, and AI-powered graph search. This could justify higher pricing tiers or upsells to large customers who see the value. - It's noteworthy that even beyond project management, **graphs are a hot topic** (e.g., knowledge graphs in search, recommendation systems in social networks, etc.). If Atlassian can say they have the first work management platform truly powered by graph technology, it sets them apart as innovators. This is an area where Atlassian can claim thought leadership, especially by leveraging Dinis's pioneering work (which, so far, few others have replicated or caught up to).

3. Integrations and Ecosystem Growth: A Jira that plays well with graph databases and graph analytics opens up new integration possibilities: - **Partnering with Graph DB Vendors:** Atlassian might partner with companies like Neo4j, TigerGraph, Amazon (Neptune), etc., to provide one-click integrations. For instance, an Atlassian Marketplace app (developed with Dinis's input or code) could let a customer mirror their Jira into a Neo4j database in real-time. This would drive value for mutual customers and possibly co-marketing opportunities. Graph database vendors would love access to Jira's install base, and Atlassian would retain relevance as those vendors approach enterprise clients. - **Third-Party Apps and Plugins:** If Atlassian exposes more graph-oriented APIs or data streams, the developer community can create apps that provide specialized visualizations or domain-specific graph analyses (e.g., risk

analysis apps, org chart generators, dependency optimizers). This could rejuvenate the Marketplace with a new class of "graph apps" – much like how the introduction of Jira Agile (now Jira Software) created an ecosystem of agile plugins initially. Dinis's open-source tools (like his visualization scripts or sync utilities) could be productized by third parties into polished apps, driving Marketplace revenue and customer choice. - **Cross-Atlassian Graph:** Atlassian could extend the graph concept across its products. Imagine linking Jira tickets to Confluence pages to Bitbucket commits all in a unified graph – something customers often try to do manually. Atlassian has already built a **Unified Search and Atlassian GraphQL API** for their cloud platform (often called the "Atlassian Graph" internally, which is an API layer) – but here we're talking about an actual *data graph* of content. If executed, a query could traverse: "Which Confluence pages were linked to requirements that were implemented in code and mentioned in Opsgenie incidents?" – answering complex cross-product questions. This would solidify Atlassian tools as an integrated ecosystem rather than separate products. Dinis's work focused on Jira, but the principles apply broadly, and his expertise could help design such cross-product schemas.

4. Customer Value: Modeling the Real World More Naturally: Many Atlassian enterprise customers use Jira to track not just software bugs, but all sorts of entities (legal contracts, inventory items, HR onboarding tasks, etc.). These use-cases push Jira beyond linear workflows and into data modeling. By officially supporting the graph approach, Atlassian would acknowledge and empower these power users: - For example, a large bank might use Jira to track audit findings, controls, policies, and compliance requirements. With graph capabilities, they could link these and ask, "which controls mitigate the most findings" or "which regulations impact our systems through multiple layers of controls?" – queries that are very hard to do today but very easy if Jira is a graph. - Another example is product development in hardware: parts, requirements, tests, and bugs all interrelate. Graph querying could find all products affected by a failing component test, etc. Jira could become the single source of truth for a **digital thread** in engineering. - In essence, treating Jira data as a graph means **closer alignment to how humans conceptualize complex systems** (networks, not flat lists). It makes Jira a more natural fit for representing real-world relationships. This deepens customer engagement because the tool adapts to their complexity, rather than forcing simplification.

5. Better Surfacing of Context and Reducing Duplication: A practical outcome of graph usage is that it becomes easier to see when something is orphaned or duplicated. If every piece of data should connect in the graph, isolated issues stand out (e.g., an incident with no linked problem ticket might need attention, or a task not linked to any story might be scope creep). Graph analysis can reveal these. Atlassian could integrate alerts or quality checks (perhaps via the automation rules engine) that use graph logic – for instance, alert if a task is not linked to a parent or if a risk has no mitigation link. This improves data hygiene and thus overall Jira data quality, which customers value especially as their Jira instances scale.

To summarize the strategic angle: **By embracing Jira as a graph database, Atlassian can transform Jira from a workflow tool into a knowledge powerhouse.** It aligns perfectly with trends in AI, enterprise knowledge graphs, and the need for better insight into complex projects. It would differentiate Atlassian in a crowded market and deepen its integration into customers' data ecosystems. Dinis Cruz's work provides a head start in this direction, with concrete examples and tools that demonstrate the possibilities.

Proposed Collaboration and Investment Initiatives

To fully realize the benefits outlined, **a collaboration between Atlassian and Dinis Cruz is proposed**. This could take several forms, but below are concrete initiatives Atlassian could undertake to leverage Dinis's expertise and contributions:

1. Strategic Advisory Role or Partnership: Atlassian can bring Dinis on as a strategic advisor or even as a temporary product consultant specifically to drive "Jira Graph" capabilities. In this role, Dinis would work with Jira's product and engineering teams to incorporate his findings into Atlassian's roadmap. Given his deep experience, he could quickly highlight low-hanging fruit (e.g., simple UI changes to better display issue links, or default link types Atlassian could ship for common uses) and also steer long-term projects (like integration with a graph query language). This engagement could be structured as a formal advisory contract or an internal collaboration (e.g., a short-term Atlassian fellow program). It sends a message that Atlassian is serious about this domain by tapping one of the leading thinkers on the topic.

2. Support and Co-Development of Open-Source Projects: Atlassian should consider **investing in Dinis's open-source tools** which align with Jira: - For **Jira Exporter/Sync (JSync)**: Atlassian could sponsor this project to maturity, ensuring it works seamlessly with Jira Cloud and Data Center editions. The outcome might be an officially supported utility (or Marketplace app) that allows customers to continuously export their Jira data to a chosen storage (database or files). Atlassian's investment (engineering time or funding) would ensure security, performance, and compatibility. In return, Atlassian could offer it as a value-add tool for enterprises or even bundle it with certain packages. - For **Project Lumos Connector**: Atlassian could collaborate with Dinis to implement the connector for a specific graph database (or a generic interface). A small financial investment (Dinis estimated on the order of £8k in his example [25]) and some developer resources could produce a working integration in short order. Atlassian could then either contribute it to open source (gaining community goodwill and feedback) or offer it as part of an enterprise toolkit. Additionally, Atlassian could engage with a Graph DB vendor in a three-way partnership to co-fund this connector, showcasing it in a joint press release (much like the hypothetical one Dinis wrote [16]). This would create buzz in both communities. - For **MGraph-DB (Memory Graph)**: If Atlassian is interested in the AI angle, they could help Dinis accelerate MGraph's development. This might involve funding specific features needed to integrate with Atlassian products or using it as a base to experiment with an "Atlassian Intelligence Graph Engine." Atlassian could also incorporate MGraph concepts into its existing infrastructure – for example, using an in-memory graph approach to power new search or recommendation features within Jira Cloud (which could be more efficient than hitting a database for every complex query). Supporting MGraph might simply be providing compute credits, minor funding, or even just recognition and encouragement through Atlassian's developer programs. It's an emerging technology that Atlassian could shape early.

3. Joint Research and Whitepapers: Atlassian and Dinis could collaborate on authoritative content (whitepapers, case studies) about Jira as a graph database. For instance, they could document a case study (perhaps anonymizing Dinis's Photobox experience or another client's success) to show the ROI of this approach. A whitepaper could detail how an organization implemented Jira as a knowledge graph and the benefits realized (faster incident resolution, better project alignment, etc.). This serves both as marketing material and as educational content for customers. Dinis's LinkedIn posts and extensive reports can serve as a starting draft for official Atlassian publications, of course refined for broader audience. By publishing this under Atlassian's banner (with Dinis as co-author), Atlassian takes thought leadership in this space. It signals to customers and investors that Atlassian is forward-thinking about new applications of its platform. It's also a low-cost, high-impact way to leverage Dinis's vision – essentially amplifying his message with Atlassian's reach.

4. Product Feature Prototypes: With Dinis's help, Atlassian could build prototypes for one or two key features to test the concept: - One idea is a **"Graph View" in Jira**. This could start as an experimental module or a plugin that, for a given issue or project, visually displays connected issues in a node-link diagram (using technology like Graphviz or Cytoscape). Dinis's PlantUML diagrams show a basic version of this; Atlassian could create an interactive version. Dinis can advise on which relationships to display and how to layout the graph meaningfully. A prototype could be tested with design partners or

internally (Atlassian's own use of Jira) to refine its usefulness. - Another is **Graph Query Language Support**. Perhaps as a Forge app extension, Atlassian could allow advanced users to run graph queries (maybe Gremlin or Cypher syntax) against their Jira data. Under the hood this could use an in-memory graph of the Jira project (maybe via MGraph). Dinis's experience writing custom queries for Jira data and addressing pitfalls (like performance, recursion limits) would be valuable in designing this. Even if it's not exposed to all end-users, having the engine to do multi-hop queries would enable many future capabilities (AI, analytics, etc.). - **Link Recommendation AI**: Using graph algorithms, Jira could recommend links. For example, if two issues have similar descriptions or are frequently updated together, Jira might suggest linking them (or even auto-link if confidence is high). This is analogous to how social networks suggest connections. Dinis's knowledge of common patterns (and perhaps using AI on the graph) could aid Atlassian in developing such features. The result is less manual effort for users to maintain the graph; it starts to maintain itself. Collaboration could involve a short project to identify where AI can auto-detect relationships in Jira data.

5. Investment (Funding and Resources): Should Atlassian see major promise here, it could consider a more direct investment: - **Acquire or Invest in IP:** If appropriate, Atlassian could acquire the intellectual property of some of Dinis's projects (or hire him full-time) to embed the technology into Atlassian's platform. For instance, acquiring OSBot-Jira or MGraph-DB if those have unique code that Atlassian wants to build on. This would ensure Atlassian fully controls the evolution of these ideas. - **Sponsor a Development Team:** Atlassian might spin up a small internal team (with Dinis as an external collaborator or advisor) to focus on "Project X" (e.g., Jira Graph Initiative) for a defined period (6–12 months). The goal could be to deliver a beta feature or integration. This is a more substantial commitment, but Atlassian has a history of incubating new features (for instance, Jira was extended with Roadmaps feature via such an initiative). The cost is justified if it leads to features that attract or retain enterprise customers. Dinis's involvement would de-risk the project because of his head-start and expertise.

6. Community Building and Evangelism: Embracing this concept also offers Atlassian a narrative to engage the community. Atlassian can collaborate with Dinis to host webinars or sessions on "Graph Databases and Jira" for the Atlassian user community. Dinis is an experienced speaker and could co-host Atlassian community events or Summit (Team) conference talks showcasing what's possible. This drives excitement and could lead to user-driven innovation (some customers may start adopting the techniques immediately once they learn them). Atlassian could then fold the best practices back into the product. Essentially, Atlassian and Dinis together would evangelize the idea of treating work management data as a graph – something that could set a trend that Atlassian leads.

In executing these initiatives, it's important to note Dinis's own enthusiasm and willingness. He has publicly expressed that he *"really wish[es] that Jira dedicated more resources to fully leveraging its ability to store data as graph nodes and edges."* [33] . In other words, he's eager for Atlassian to take this seriously, and his sharing of research is almost an open invitation. By responding to that call, Atlassian not only gains technically, but also earns goodwill from thought leaders and early adopters who see that Atlassian listens and innovates.

Conclusion

Dinis Cruz's body of work illuminates a compelling path forward for Atlassian Jira: **evolve it into a first-class graph-powered platform**. This proposal has outlined how Dinis's extensive background with Jira and his pioneering research into graph databases converge into a vision that aligns with Atlassian's strategic interests. Jira is already entrenched in the operations of thousands of companies – enhancing

it with graph capabilities would deepen its value and unlock new use cases from high-level strategic planning to day-to-day knowledge discovery.

For Atlassian, the timing is perfect. The industry is gravitating towards knowledge graphs and AI-driven insights. By investing in this direction – guided by someone like Dinis who has been ahead of the curve – Atlassian can leapfrog competitors and address latent customer needs. The collaboration could start modestly (funding a connector or integrating an open-source tool) and grow into a core part of Jira's roadmap.

In conclusion, partnering with Dinis Cruz offers Atlassian a rare opportunity to incorporate **years of innovative R&D at minimal cost and risk**. The payoff is a Jira that not only tracks work, but truly understands and maps the work – a Jira that helps organizations see the forest of their projects, not just the trees. This is the vision Dinis has championed, and with Atlassian's support, it can become a reality, driving the next chapter of Jira's leadership in the market.

Sources:

- Cruz, Dinis. *LinkedIn Post – Using Jira as a Graph DB.* (2025) – Dinis's overview of treating Jira issues as nodes and links as edges [34] [12].
- Cruz, Dinis. *"Jira's Unique Value as a Graph Database" – LinkedIn Document.* (2025) – In-depth research comparing Jira to Neo4j, Neptune, etc., and highlighting Jira's strengths and solvable limitations [2].
- Cruz, Dinis. *Graph Database Techniques in Jira – Issue Type and Project Design.* (2025) – Technical paper on structuring Jira projects such that each represents a node type in a graph [7] [8].
- Cruz, Dinis. *Graph Database Techniques in Jira – Issue Link Types.* (2025) – Explains how Jira's bidirectional issue link design enables rich relationships and ontologies in the data [35].
- LinkedIn Post by Dinis Cruz – *"Project Lumos: Jira-to-GraphDB XYZ Connector".* (2025) – Working Backwards style proposal for an £8k investment to build a serverless Jira-to-GraphDB integration [25] [36].
- Slideshare (OWASP London, Apr 2019) – *"Creating a Graph-Based Security Organisation".* – Case study of Photobox's Jira implementation as a graph for risk management (notes that the risk workflow and Jira were refactored into a graph-backed system, with visualizations and automation) [1].
- Slideshare (Open Security Summit 2018) – *"Thinking in Graphs v1.0".* – Presentation by Dinis Cruz (Photobox CISO) illustrating use of Neo4j and Jira to model security incidents and projects as graphs [6].
- Cruz, Dinis – Various LinkedIn updates in 2025 demonstrating Jira graph examples (PlantUML visualizations, "paths from CEO" graph) and discussing autogenerated decision documents [20] [22].
- GitHub – *OWASP Security Bot (OSBot) Repositories.* – Open-source projects by Dinis including **OSBot-Jira** (automation scripts) and **MGraph-DB** (Memory-first Graph Database) [5] [27].
- Internal Atlassian vision documents (hypothetical, based on context) – Aligning Jira's development with knowledge graph and AI trends (no direct citation, strategic rationale derived from industry context and Dinis's commentary).

---

[1] [10] [11] Creating a graph based security organisation - Apr 2019 (OWASP London chapter meeting) | PPT
https://www.slideshare.net/slideshow/creating-a-graph-based-security-organisation-apr-2019-owasp-london-chapter-meeting/140239010

2 17 18 Jira unique value as a GraphDB | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_jira-unique-value-as-a-graphdb-activity-7293409497316745218-Xp9i

3 23 24 project plan - jira export to s3 and git | Dinis Cruz
https://www.linkedin.com/feed/update/urn:li:activity:7294766710589456385/

4 15 16 25 26 30 36 Project Lumos | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_project-lumos-activity-7295968679869972480-NYGC

5 27 28 29 31 32 Building Semantic Knowledge Graphs with LLMs: Inside MyFeeds.ai's Multi-Phase Architecture
https://www.linkedin.com/pulse/building-semantic-knowledge-graphs-llms-inside-myfeedsais-dinis-cruz-jub5e

6 Thinking in graphs v1.0 | PPT
https://www.slideshare.net/slideshow/thinking-in-graphs-v10/101283923

7 8 Jira Issue Type and Project Design | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_jira-issue-type-and-project-design-activity-7295568356135301121-Gahn

9 35 Jira - Issue Link Types | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_jira-issue-link-types-activity-7295587710956695552-24s2

12 33 34 Jira as Graph DB | Dinis Cruz | 10 comments
https://www.linkedin.com/feed/update/urn:li:activity:7293099257547354112/

13 Jira – Workflow Design and Customization | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_jira-workflow-design-and-customization-activity-7295579432453394432-swsn

14 Storing timestamps in a graph database | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_storing-timestamps-in-a-graph-database-activity-7294775813302829058-K5ir/

19 Jira as Graph DB | Dinis Cruz | 10 comments
https://www.linkedin.com/posts/diniscruz_jira-as-graph-db-activity-7293099257547354112-z4xs

20 Ahh, I missed making these graphs! :) Creating graphs like this (from... | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_ahh-i-missed-making-these-graphs-creating-activity-7293630633333915648-kiD4

21 Jira as Graph DB - First Visualisation | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_jira-as-graph-db-first-visualisation-activity-7293634409763663873-trGn

22 How to capture a decision with Jira | Dinis Cruz posted on the topic | LinkedIn
https://www.linkedin.com/posts/diniscruz_architecture-decisions-shape-the-future-of-activity-7305258956967297026-ADX5